

Software Design and Engineering

1.System Design

2. Introduction to High level Design

3. What is Low Level Design or LLD?

4. System Design Life Cycle (SDLC)

5.Design Patterns Tutorial

6.Unified Modeling Language (UML) Diagrams

7.Scalability in System Design

8. Software Engineering Tutorial

Taken from

<https://www.geeksforgeeks.org/system-design/getting-started-with-system-design/>

<https://www.geeksforgeeks.org/system-design/what-is-high-level-design-learn-system-design/>

<https://www.geeksforgeeks.org/system-design/what-is-low-level-design-or-lll-learn-system-design/>

<https://www.geeksforgeeks.org/system-design/system-design-life-cycle-phases-models-and-use-cases/>

<https://www.geeksforgeeks.org/system-design/what-is-scalability/>

<https://www.geeksforgeeks.org/system-design/software-design-patterns/>

<https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/>

<https://www.geeksforgeeks.org/system-design/how-to-crack-system-design-round-in-interviews/>

SW Engineering

<https://www.geeksforgeeks.org/software-engineering/software-engineering/>

<https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/>

DSA

<https://www.geeksforgeeks.org/dsa/dsa-tutorial-learn-data-structures-and-algorithms/>

Database

<https://www.geeksforgeeks.org/system-design/complete-reference-to-databases-in-designing-systems/>

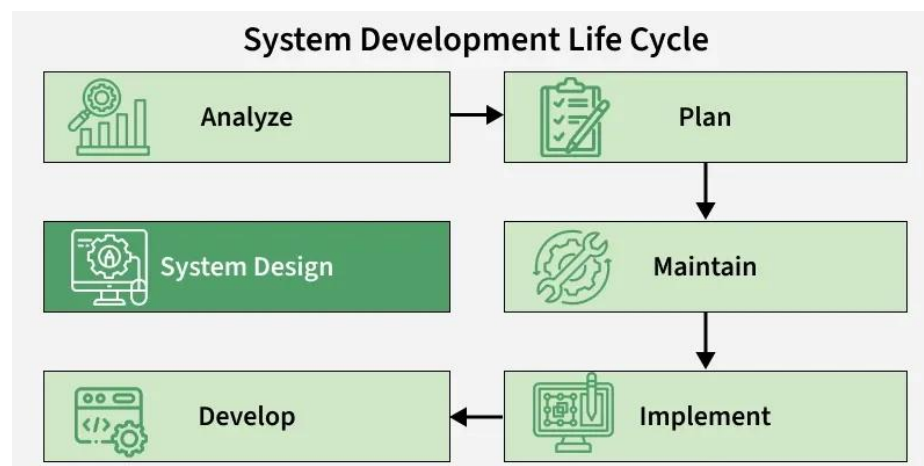
1.System Design

System design is the process of planning, structuring and defining the architecture of Software System.

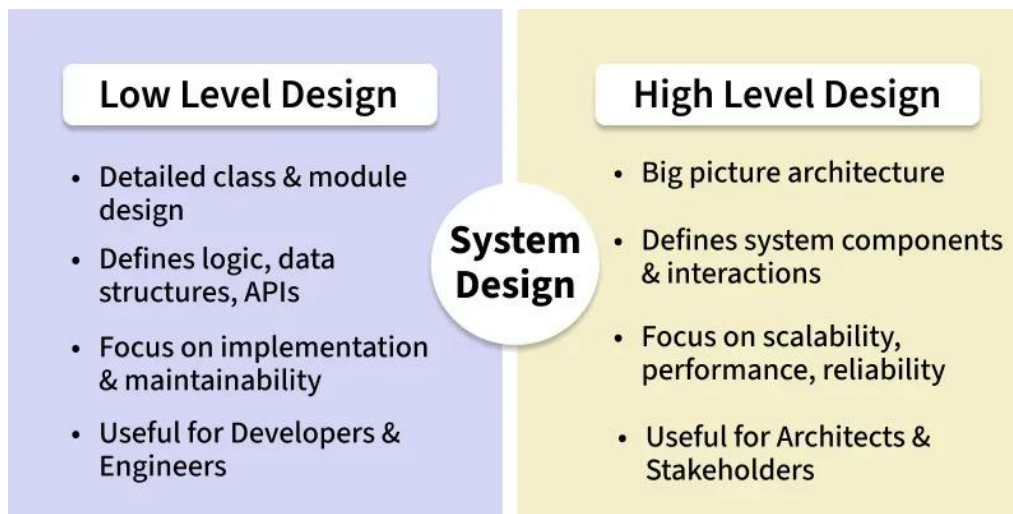
- Involves translating user requirements into a detailed blueprint that guides the implementation phase.
- The goal is to create a **well-organized** and **efficient structure** that meets the intended purpose while considering factors like **scalability**, **maintainability**, and **performance**.

System Design in SDLC

- In System Design Life Cycle, without the **designing phase**, one cannot jump to the implementation or the testing part.
- System Design is a vital step and also provides the backbone to handle exceptional scenarios because it represents the **business logic** of the software.



System Design can be divided into two complementary parts



High-Level Design (HLD)

It lays out the overall architecture of the system — how major components interact, what services or modules will exist, and how data flows among them.

- Gives you the big picture: how the system fits together, its core structure, and major decisions.
- Done by architects, stakeholders and managers

Prerequisite Technical Knowledge for HLD

- Basic Coding Skills (Data Structures and Algorithms)
- Compared to Low Level Design, High Level Design is typically done by more senior people who have hands-on experience on software projects.
- Knowing the roles of components like databases (SQL and NoSQL), caches (Redis, Memcached, CDNs), and APIs.
- Low Level Design (Object Oriented Programming and Design Patterns)
- In-depth understanding of Functional Requirements (What the system must do e.g., user registration, streaming) and Non-Functional Requirements (How the system should perform—scalability, latency, availability, security, etc)
- Networking and Security Fundamentals like DNS, protocols (TCP/UDP, HTTP, WebSockets), OAuth, JWT, TLS/SSL, rate-limiting, API security, and basic DDOS protection.

- Message queues and streaming tools like Kafka or RabbitMQ.
- Knowledge of Microservices vs. Monoliths (When to split services and how to manage dependencies), fault tolerance, fallback strategies, redundancy, Load Balancer Types & Algorithms.
- Observability tools like Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana) and Alerting systems (e.g., PagerDuty)

Topics Covered in HLD:

- **System architecture overview:** Defines the major components, modules, and how they interact (e.g., services, queues, databases) .
- **Data flow and component interaction:** Illustrates how data moves between modules, along with key integrations and interfaces.
- **Technology stack and infrastructure:** High-level decisions on frameworks, platforms, hardware, databases, and hosting setups.
- **Module responsibilities:** Describes what each module does and how they relate to one another .
- **Performance & trade-offs:** Includes design trade-offs, performance considerations, scalability, security, cost and other non-functional factors .
- **Artifacts:** Commonly includes architecture diagrams, component and deployment diagrams, data flow diagrams, and possibly ER/DB schematic overviews.

Real World Examples of HLD Decisions

- Netflix transitioned their entire backend from a monolith to microservices (starting with encoding and UI services), completing the migration by 2011 to scale rapidly during high-load events like holiday seasons.
- Uber adopted an event-driven architecture where ride requests, location updates, and fare changes emit events that trigger real-time systems like driver matching, billing, and dynamic pricing.
- Twitter deployed a load-balanced architecture with caching of trending topics and tweets to quickly serve millions of users and handle real-time data flows efficiently.

Low-Level Design (LLD)

It covers how each part works & is implemented internally

- Gives developers a clear, actionable blueprint to build each component.
- Once a High Level Design is built by stakeholders, it is the job of senior developers and designers to create a low level design.

Prerequisites / What You Should Know First:

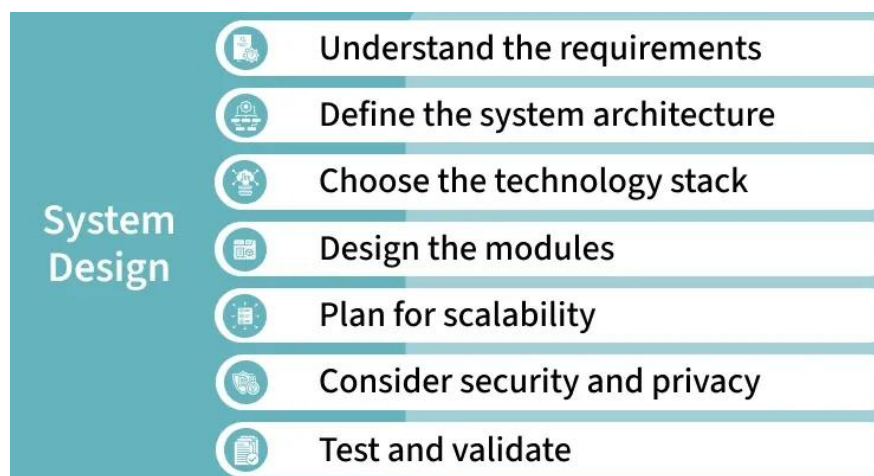
- Basic Coding Skills (Data Structures and Algorithms)
- Strong grasp of OOP concepts (Encapsulation, inheritance, polymorphism & abstraction)

Topics Covered:

- **Component/module breakdown:** Detailed internal logic for each module—with class responsibilities, methods, attributes, interactions
- **Database schema & structure:** Designing tables, keys, indexes, relationships with SQL/NoSQL refinements
- **API & interface definitions:** Precise request/response formats, error codes, methods, endpoints, and internal interfacing
- **Error handling & validation logic:** Define how each module manages invalid inputs, failures, edge cases, and logging
- **Design patterns & SOLID:** Implement design patterns and solid principles to ensure clean, extensible, maintainable code
- **UML and pseudocode artifacts:** Class diagrams, sequence diagrams, pseudocode or flowcharts to clarify logic paths and method calls

Steps for getting started with System Design

Here are some steps to get started with system design:



- **Understand Requirements**
Gather and analyze business needs by consulting stakeholders, users, and documentation.
- **Define Architecture**
Identify key system components and how they interact (e.g., services, APIs, databases).
- **Choose Tech Stack**
Select appropriate languages, databases, frameworks, and tools based on requirements.
- **Design Modules**
Break the system into modules, defining their responsibilities and data flow.
- **Plan for Scalability**
Design with growth in mind—anticipate load, optimize bottlenecks, and use scalable patterns.
- **Ensure Security & Privacy**
Identify risks and implement measures like authentication, encryption, and data protection.
- **Test & Validate**
Write test cases and simulate real-world usage to ensure the system meets requirements.

Overall, system design is a complex process that requires careful planning and attention to detail. By following these steps, you can create a system design that meets the needs of your users and your business.

Tips and Tricks to solve System Design Problem

- When you are given a System Design Problem, you should approach it in a planned manner.
- Initially, the Problem may look huge, and one can easily get confused about how to start solving it.
- And moreover, there is no fixed solution while you are designing a system.
- There is more than one way to reach the Solution.

So let's discuss how one should start with solving a Design Problem given in an Interview.

Approaching a Design Problem

1. Breaking Down the Problem

When you are given a Design Problem start breaking it down into small components. These components can be Services or Features which you need to implement in the System.

- Initially, a System given to be designed can have a large number of features and you are not expected to design everything if it's an Interview.
- Ask your interviewer about the Features you are planning to put in your system. Is there anything else you should be putting there? Any Feature? Any Service? ... Ask!

2. Communicating your Ideas

Communicate well with the Interviewer:

- While designing the system keep him in the loop.
- Discuss your process out loud.
- Try to demonstrate your design clearly on the whiteboard with flowcharts and diagrams.
- Describe your ideas to your interviewer, how you have planned to tackle the problem of scalability, how you will be designing your database, and many others.

3. Assumptions that make sense

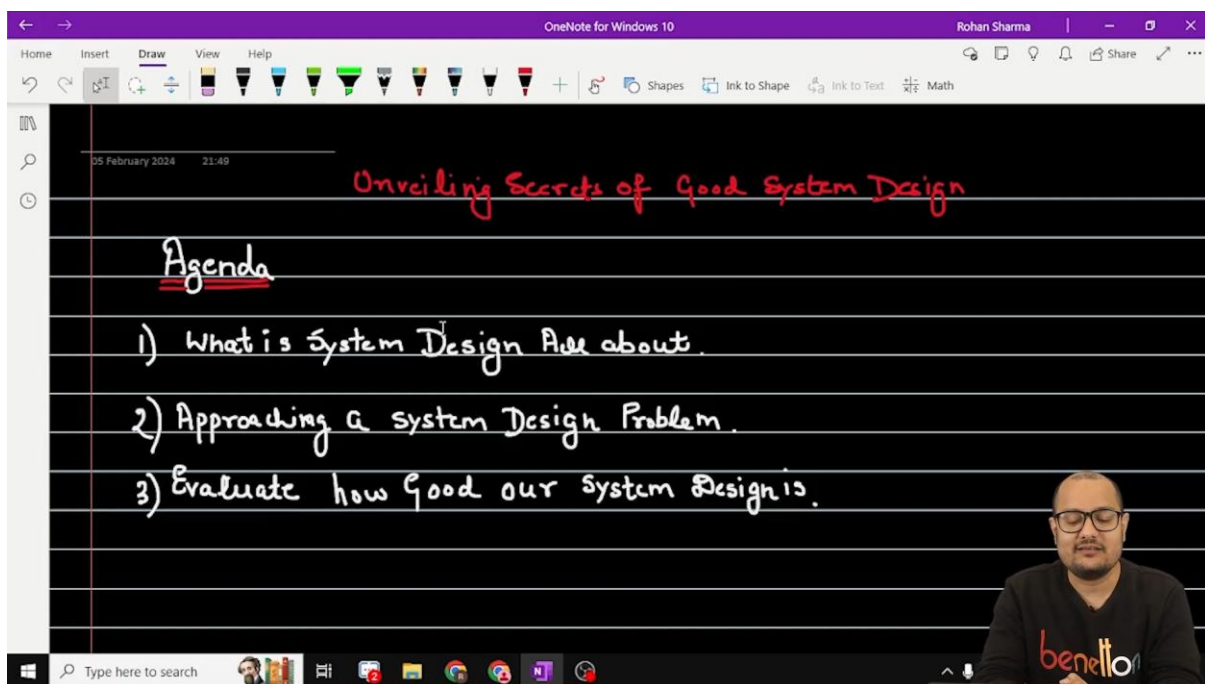
Make some reasonable assumptions while you are designing the System.

Suppose you have to assume the number of requests the system will be processed per day, the number of database calls made in a month, or the efficiency rate of your Caching System. These are some of the numbers you need to assume while designing. Try to keep these numbers as reasonable as possible. Back up your assumption with some solid facts and figures.

Important points to consider when designing a software system:

1. **Scalability:** The system should be designed to handle increased loads and be able to scale horizontally or vertically as needed.
2. **Performance:** The system should be designed to perform efficiently and effectively, with minimal latency and response time.

3. **Reliability:** The system should be reliable and available, with minimal downtime or system failures.
4. **Security:** The system should be designed with security in mind, including measures to prevent unauthorized access and protect sensitive data.
5. **Maintainability:** The system should be designed to be easy to maintain and update, with clear documentation and well-organized code.
6. **Interoperability:** The system should be designed to work seamlessly with other systems and components, with clear and well-defined interfaces.
7. **Usability:** The system should be designed to be user-friendly and intuitive, with a clear and consistent user interface.
8. **Cost-effectiveness:** The system should be designed to be cost-effective, with a focus on minimizing development and operational costs while still meeting the requirements.



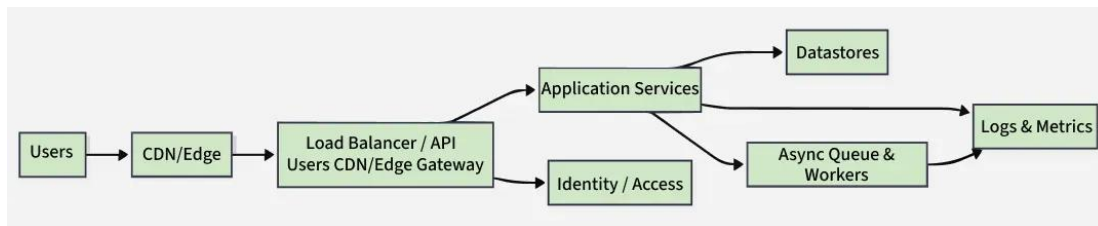
2. Introduction to High level Design

High-level design or HLD is an initial step in the development of applications where the overall structure of a system is planned.

- Focuses mainly on how different components of the system work together without getting to know about internal coding and implementation.

- Helps everyone involved in the project to understand the goals and ensures good communication during development.
- Crucial for developers, architects, and product managers because it allows them to make sure that all stakeholders are aligned with the project objectives. That's why it is also known as macro-level design.

The common request flow most applications follow from user to data and back.



- **Users - Edge/Gateway:** CDN/Edge handles cache, TLS, basic DDoS; LB/API Gateway routes.
- **Gateway - Services:** App layer runs auth/logic and fans out to deps.
- **Services - Data:** Read/write to SQL/NoSQL/object/storage/search; cache hot reads; persist writes.
- **Services - Async:** Offload emails/webhooks/reindex/heavy jobs to queues; workers update later.
- **Auth & Observability:** Auth at gateway/services; emit logs, metrics, traces for monitoring/alerts.
- **Response - User:** Reply goes back via gateway/edge; update caches for faster next hits.

Components of High-Level Design

Understanding the components of high-level design is very important for creating effective systems that meet user needs and technical requirements.

Below are the main components of high-level design:

1. **System Architecture:** System architecture is an overview of the entire system that represents the structure and the relationships between various components. It helps to visually represent how different parts interact and function.
2. **Modules and Components:** High-level design breaks down the systems into modules or components, each with specific roles and responsibilities, and has a distinct function that contributes to the entire system, helping in developing an efficient system.

3. **Data Flow Diagrams (DFDs):** Data Flow Diagrams demonstrate the data movement within the system. They help to understand how information is processed and pass from one end to another.
4. **Interface Design:** It includes the design of application programming interfaces (APIs) for system integration and user interfaces (UIs) for user interaction, ensuring seamless functionality and communication between components.
5. **Technology Stack:** The technology stack is various technologies and tools that will be used in the development of the system. This includes programming languages, frameworks, and databases.
6. **Deployment Architecture:** It includes how the system will be hosted and accessed. It includes server configurations, cloud infrastructure, and network considerations.

What is High-Level Design Document?

The HLD document consists of data flows, flowcharts, and data structures to help developers understand and implement how the current system is being designed intentionally to function.

- This document is responsible for explaining the connections between system components and operations, which depict the logic.
- The architecture design needed (for the system's functionality and flow) for each and every module of the system as per the functional requirements.

Purpose and Characteristics of High-Level Design

The purpose of this High-Level Design (HLD) is to add the necessary detailed description to represent a suitable model. This is designed to help with operational requirements and will help to understand how the modules interact. Basically, HLD is a technical representation of functional requirements and the flow of information across components. Characteristics of high-level design include

- A diagram representing each design aspect is included in the HLD (which is based on business requirements and anticipated results).
- Description of hardware, software interfaces, and also user interfaces.
- The workflow of the user's typical process is detailed, along with performance specifications.

How HLD is Different from LLD

High-level design, or HLD, is a general system design where we do tradeoffs between different frameworks, components, and different databases, and we choose the best considering what the business needs and how the system should work, both in terms of functional and non-functional aspects.

Whereas LLD (low-level design) translates the HLD into smaller and more specific details, it includes class diagrams, methods, data structures, and algorithms, focusing on how each part will be implemented.

Further Read: [Differences between High-Level Design \(HLD\) and Low-Level Design \(LLD\)](#)

After having an adequate understanding of HLD and how it is different from LLD. Let us now discuss the HLD roadmap, which is shown below as an infographic:

How to Design Scalable High-Level Design (HLD) Systems

Now, in order to design any high-level system, certain terms are listed in a sequential manner so as to connect the dots in order to understand. Roadmaps are very important for working professionals because these help us to get a complete, binding understanding of how services in real-time are getting scaled at a high level.



1. Capacity Estimation

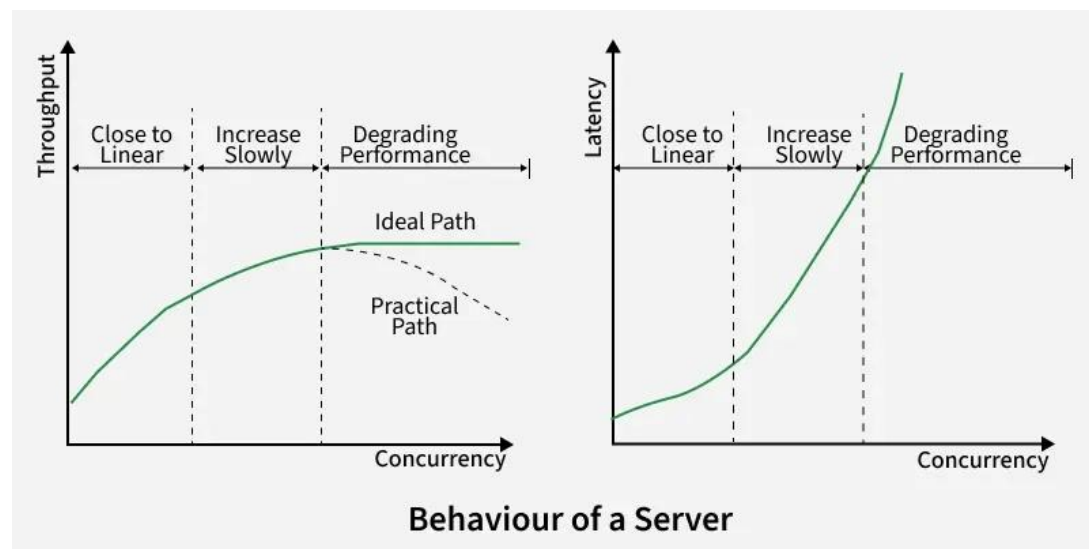
Capacity estimation in system design involves predicting the resources (such as processing power, memory, and bandwidth) required to meet the expected workload. It ensures that a system can handle current and future demands

efficiently, helping in the proper allocation of resources and preventing performance bottlenecks.

For example:

Twitter, which is recently in newsfeeds these days, here at high-level designing, we need to make sure tweets of popular influencers are reaching out to millions of people, so how do we need to scale our system so service should not be hampered?

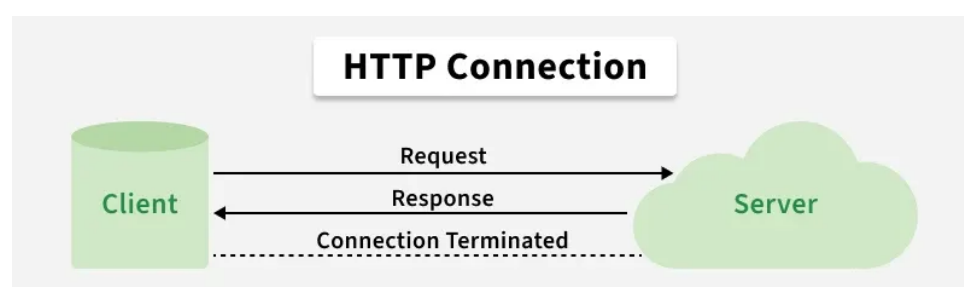
We have also shown below the characteristic behavior of a server, which is measured across throughput and latency within the system.



2. HTTP and HTTPS and Their Methods

HTTP (HyperText Transfer Protocol) is used to transfer the data over the web, which enables the communication between clients and servers that helps users to request resources like HTML pages, messages, videos, or images, while [HTTPS](#) (HyperText Transfer Protocol Secure) is an extension of HTTP that adds an extra layer of security through SSL/TLS encryption. Methods of HTTP include GET, PUT, and POST.

Tip: *These are also important as per interview perspective as constraint bounds in interviews in API over when to use what is very important.*



Tip: There are 2 ways to send data from server to client:

- Using Websockets
- Using Polling

3. Web Sockets

There are two ways to send data from server to clients, which are websockets and polling. In a [client-server architecture](#), we are sending a request to the server, and the server sends it back, and so in this way communication takes place. But in designing the system, the biggest problem we face is whether we can implement the client-server model or [peer-to-peer model](#).



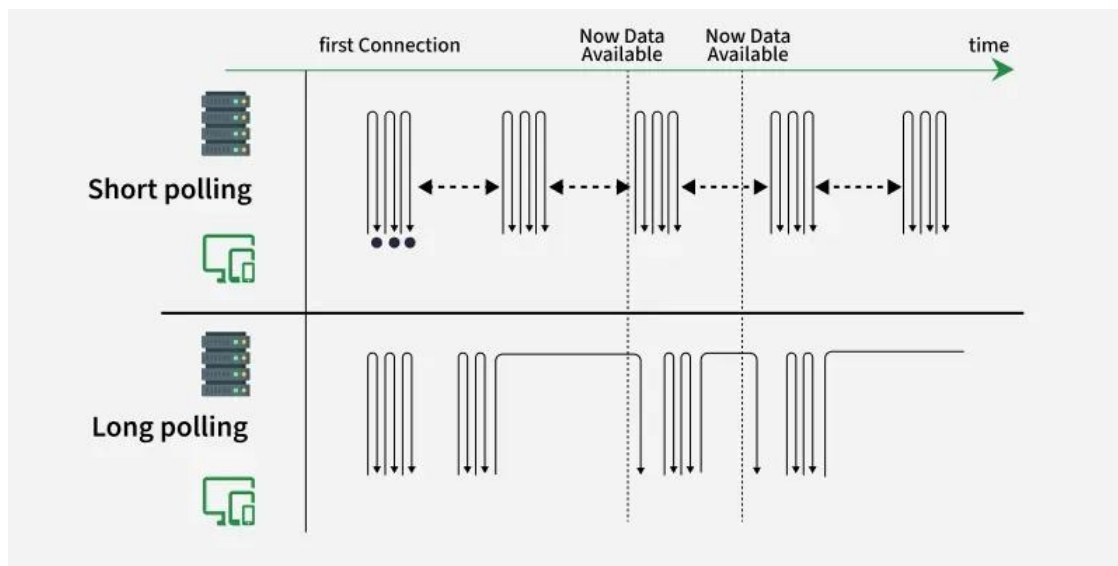
- Here WebSockets play a crucial role in solving problems and enabling us to have real-time communication.
- WebSocket is a full-duplex protocol as it allows the application to send and receive data at the same time.
- They are more or less used in every system to be designed because we cannot let our system wait for a response.
- They are very useful in real-time web applications, gaming applications, and chat applications.

4. Polling

Another way of sending and receiving data from a server, just like we do with the above case of web sockets, is polling. There are mainly 2 types of polling:

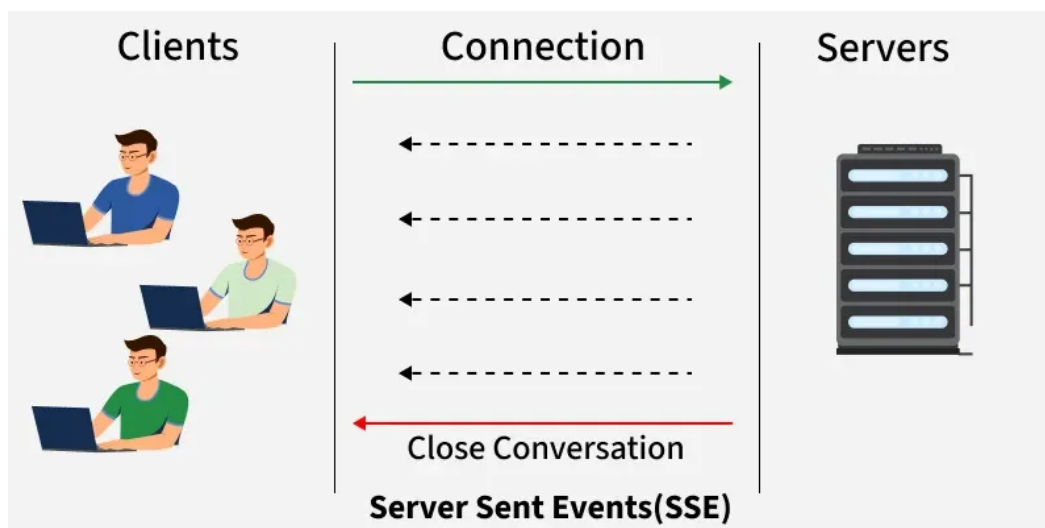
- **In short polling**, the client sends a request to the server, and the server immediately sends data at fixed intervals, whether the data is updated or not.
- **In long polling**, the server waits for the client's request to respond and responds only and only if some new data is available or if some data is being updated.

Tip: Long polling is preferred over short polling because lesser number of requests are sent in a system.



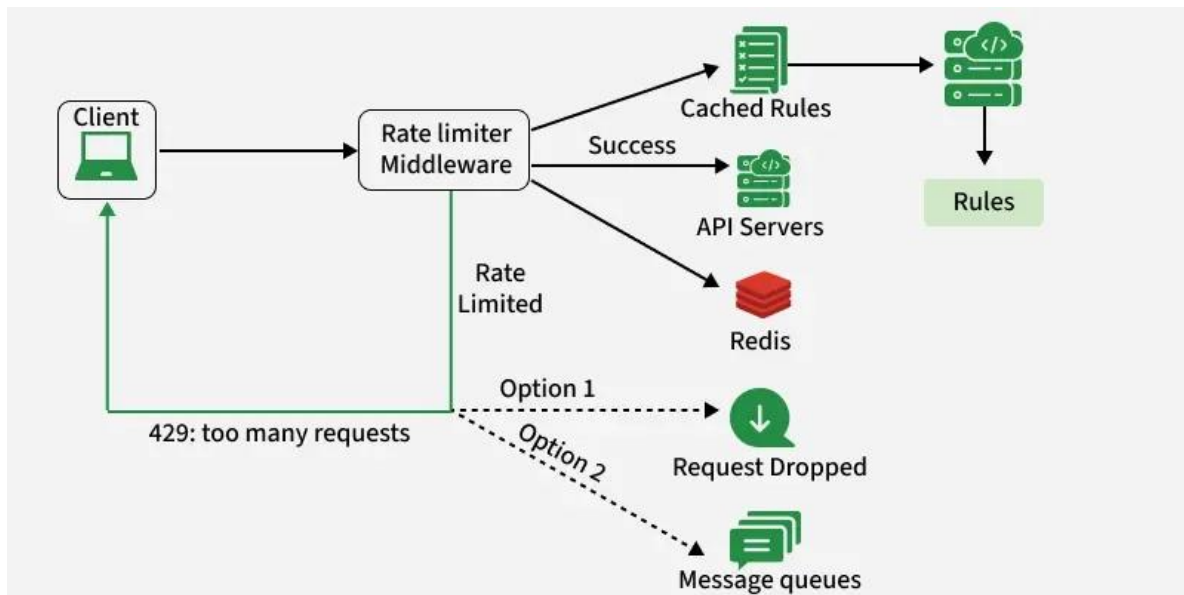
5. Server-Sent Events(SSE)

It is purposely built as **one-way communication** from servers to clients in specific design systems. SSE is a technology that helps to push real-time updates from the server to the clients over HTTP connections. SSE enables the server to send data automatically as it becomes available. Examples of SSE include real-time streaming.



6. Rate limiting

A rate limiter restricts the number of events that can be done in a timeframe by restricting the number of requests a sender can send in a given period of time. Here, once the threshold limit is reached, it now further blocks the incoming requests, as can be seen below in the media as follows:



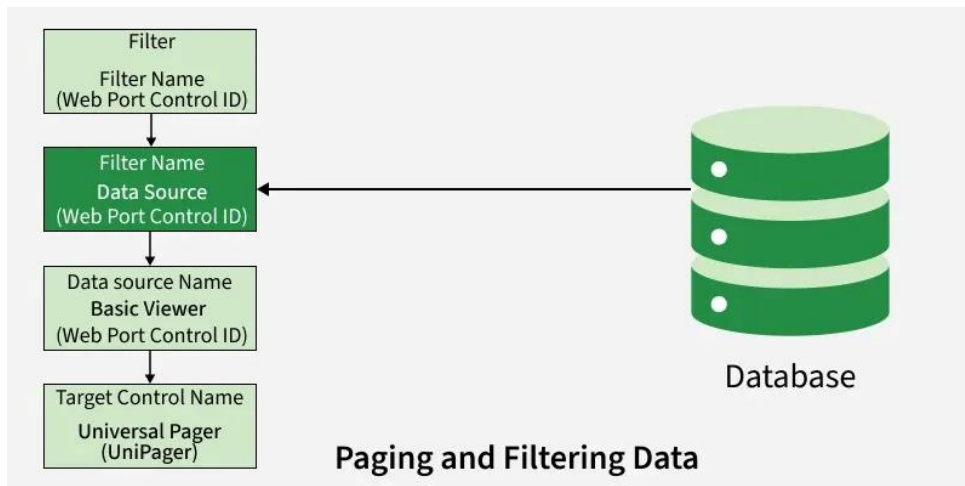
7. Resiliency

No matter how great a system we design, there is always a chance of faults and failure, which could be because of hardware issues or software issues (such as running low on memory), or there can be some human error. In such cases we need to provide resiliency through replication, redundancy, and availability.

Note: System design interviews start with open-ended designing of a specific system, which later is bounded with constraints at multiple levels. In order to clear dealing with these bounds, in layman's language, it is known as resiliency via implementing common tradeoffs in the system. Hence, providing resiliency is very crucial in designing complex system designs and also in clearing interviews.

8. Paging

Paging in high-level design (HLD) refers to the method of dividing large datasets or content into smaller, manageable pages. This approach improves user experience by allowing users to load data incrementally, reducing initial load times and enhancing performance.



9. Logging

A log file records details of events occurring in a software application. The details may consist of microservices, transactions, service actions, or anything helpful to debug the flow of an event in the system. Logging is crucial to monitor the application's flow. This can also be useful for tracking the health and performance of a distributed system, as well as for debugging issues that may arise. There are several approaches to implementing distributed logging, including:

- Using a [centralized logging service](#)
- Using a [distributed logging system](#)
- Using a [cloud-based logging service](#)

High Level Design

14 September 2022 / 09:41

2. High level design:

- The expectation here is to identify major components of the system in agreement with the interviewer
- Draw the components and their interaction, in every step ask for feedback from the interviewer
- Ensure that initial design is small (fits into a page / half of the whiteboard)
- The interviewer could ask to detail out certain critical components.
- Go through certain concrete use cases, it can help unfurl the edge cases which may not have been considered.

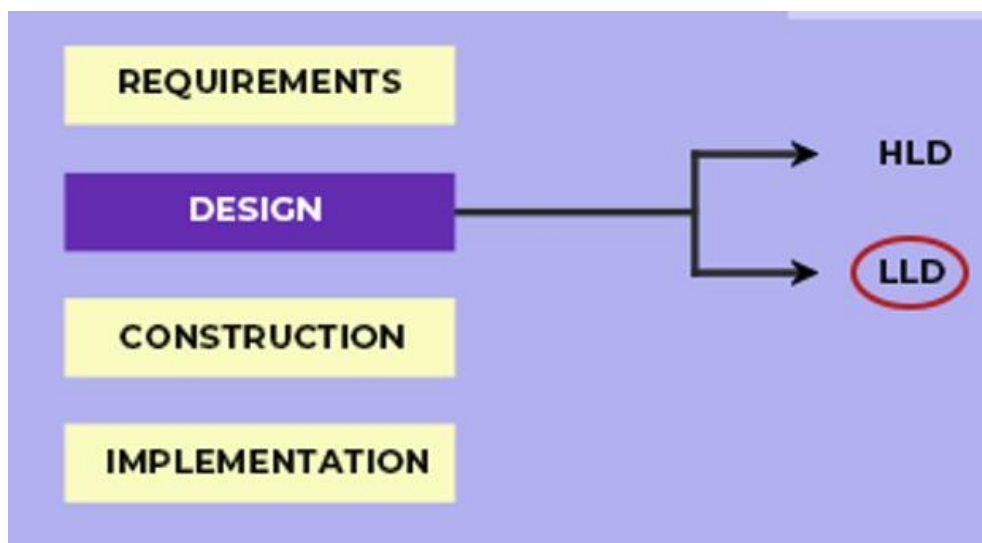
<https://www.geeksforgeeks.org/courses/mastering-system-design-low-level-to-high-level-solutions>

3. What is Low Level Design or LLD?

<https://www.geeksforgeeks.org/system-design/what-is-low-level-design-or-lll-learn-system-design/>

Low-Level Design (LLD) plays a crucial role in software development, transforming high-level abstract concepts into detailed, actionable components that developers can use to build the system.

- LLD is the blueprint that guides developers on how to implement specific components of a system, such as classes, methods, algorithms, and data structures.
- Whether we are working on a microservice architecture, a web application, or a mobile app, understanding LLD is essential for building scalable, maintainable, and efficient systems.



How is LLD different from HLD?

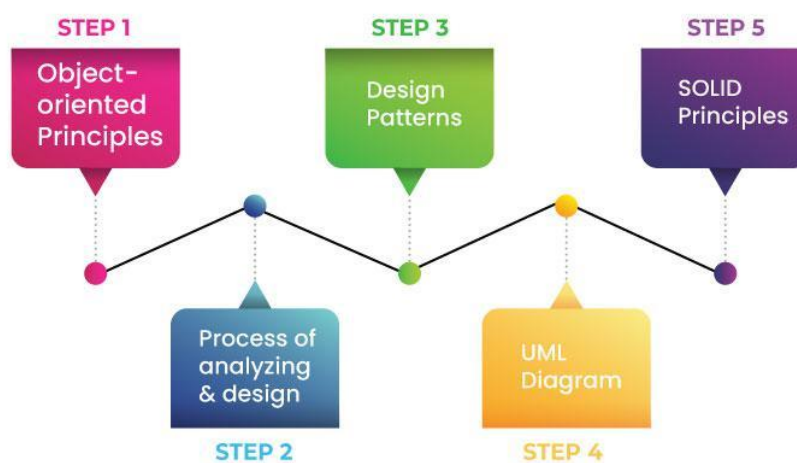
While both **HLD** (**High-Level Design**) and **LLD** (**Low-Level Design**) are critical in system architecture, they serve different purposes.

1. High-Level Design (HLD): Focuses on the **overall architecture** of the system. It addresses questions such as which frameworks to use, what databases are suitable, how to integrate different components, and how the system will function at a broader level.

2. Low-Level Design (LLD): Once the high-level structure is in place, **LLD takes over** by focusing on specific **components, modules, and interactions**. It provides detailed design diagrams (like UML diagrams) and breaks down **how each component** should behave, how it will interact with others, and what algorithms and data structures will be used.

Concepts Needed to Create a LLD from given HLD

For HLD to LLD conversion, we generally use [Unified Modelling Language \(UML\) diagrams](#). Adding to these diagrams we use OOP principles and [SOLID principles](#) and design patterns while designing.



Roadmap to Low Level Design(LLD)



Step 1. Understanding Object-Oriented Principles

The user requirement is processed by using concepts of OOPS programming. OOP concepts serve as the foundation for LLD, and having a deep understanding of them will help you design maintainable and scalable software components. Hence it is recommended to have a strong grip on **Object-Oriented Programming** principles such as:

- **Encapsulation:** Bundling data and the methods that operate on that data within one unit.
- **Inheritance:** Mechanism where a new class can inherit the properties and methods of an existing class.
- **Polymorphism:** Ability of different classes to respond to the same method in different ways.
- **Abstraction:** Hiding the complex implementation details while showing only the essential features of an object.

Step 2. Analyzing and Designing Components

LLD requires you to **analyze real-world problems** and break them down into **object-world problems** using OOP concepts. This is a critical step where **real-world entities** are modeled into **objects and classes**.

You should focus on:

- **Identifying classes and objects** based on the system requirements.
- **Determining relationships** (like associations, inheritance, etc.) between different entities.
- Using **SOLID principles** to ensure your design is clean, maintainable, and scalable.

Step 3. Implementing Design Patterns

Now the implementation of our above object oriented problem is carried out with the help of design patterns. Design patterns are **reusable solutions** to common problems encountered in software design. They provide a **structured approach** to design by capturing best practices and proven solutions, making it easier to develop scalable, maintainable, and efficient software. By using these patterns, developers can solve problems more effectively while adhering to best practices.

- **Creational Patterns** (e.g., Singleton, Factory): These patterns deal with object creation mechanisms, trying to create objects in a way that is appropriate to the situation.
- **Behavioral Patterns** (e.g., Observer, Strategy): Focus on communication between objects and how they interact with each other.
- **Structural Patterns** (e.g., Adapter, Composite): These patterns are concerned with simplifying the structure of the system and its components.

Step 4. Use of UML Diagram in LLD

Unified Modeling Language is a visual representation used to design and model the system. UML diagrams play an important role in converting HLD to LLD. They provide a proper and clear **visual representation** of the components and their relationships, which helps developers significantly. Some of the most important UML diagrams used in LLD include:

- **Class Diagrams**: Represent the structure of the system in terms of its classes and the relationships between them.

- **Sequence Diagrams:** Illustrate how objects interact over time, showing the sequence of method calls.
- **Activity Diagrams:** Show the workflow or activities of a system component.
- **State Diagrams:** Represent the different states of a component or object and the transitions between these states.
- **Use Case Diagrams:** Define the functional requirements of the system by showing different user interactions.

These diagrams play a significant role in **LLD**, providing a **clear visual representation** of the components and their interactions.

5. Implementing SOLID Principles

These are sets of 5 principles(rules) that are strictly followed as per requirements of the system or requirements for optimal designing. In order to write scalable, flexible, maintainable, and reusable code:

1. **Single-responsibility principle (SRP)**
2. **Open-closed principle (OCP)**
3. **Liskov's Substitution Principle(LSP)**
4. **Interface Segregation Principle (ISP)**
5. **Dependency Inversion Principle (DIP)**

It's important to keep in mind that SOLID principles are just guidelines and not strict rules to be followed. The key is to strike a balance between following these principles and considering the specific needs and constraints of your business requirement.

Benefits of Low-Level Design(LLD)

Low-Level Design offers a **detailed and structured approach** to building software systems, providing multiple benefits:

- **Clear Component Functionality:** LLD provides a detailed plan for how each part of the software will work, making development and debugging easier.
- **Scalability and Flexibility:** It provides a well-structured design, which makes simpler to update or fix parts of the system without affecting the entire software.

- **Improved Communication:** LLD helps team members communicate more effectively because everyone has a clear understanding of how components are working.
- **Cleaner Code:** Following the design principles in LLD leads to more cleaner, organized code, making it less prone to errors.
- **Faster Coding:** A proper LLD speeds up the coding process because developers can follow the detailed plan made earlier.

Best Practices for Low-Level Design(LLD)

Below are some of the best practices to form a good Low-Level Design:

- **Modular Design:** Always try to break down the system into small, independent components that contain specific functionalities.
- **Clear Interfaces:** Clearly define the interfaces for each component, which should include methods, inputs, outputs. This helps in maintaining proper communication between components
- **Use Design Patterns:** Include OOPS Principles to promote code reusability, flexibility and maintainability.
- **Adopt SOLID Principles:** Follow Solid Principles which will lead to more robust and maintainable design.
- **Error Handling:** plan in advance for error handling and validation of the system by including validation checks in the design

Roadmap to Learn LLD

To master Low-Level Design, follow this structured learning path:

1. Object-Oriented and Design Principles

- [Object-Oriented Programming \(OOPS\) Concepts](#)
- [Object-Oriented Analysis and Design](#)
- [SOLID Principles](#)

2. Design Patterns

- [Creational Design Patterns](#)
- [Behavioral Design Patterns](#)
- [Structural Design Patterns](#)
- [When to choose which design pattern?](#)

3. UML and Modeling in LLD

- [Class Diagrams](#)
- [Sequence Diagrams](#)
- [Activity Diagrams](#)
- [State Diagrams](#)
- [Use Case Diagrams](#)
- [Object Diagrams](#)

4. LLD Best Practices

- [Data Structures and Algorithms for System Design](#)
- [Essential Security Measures in System Design](#)
- [Introduction to Modularity and Interfaces](#)
- [Difference between LLD and HLD](#)
- [Performance vs Scalability](#)
- [Speed vs. Quality](#)
- [Coding Standards Best Practices](#)

5. LLD Case Studies

- [Elevator System Low-Level Design](#)
- [Tic-Tac-Toe Low-Level Design](#)
- [Amazon Low Level Design](#)
- [Vending Machine Low Level Design](#)

6. Interview Preparation

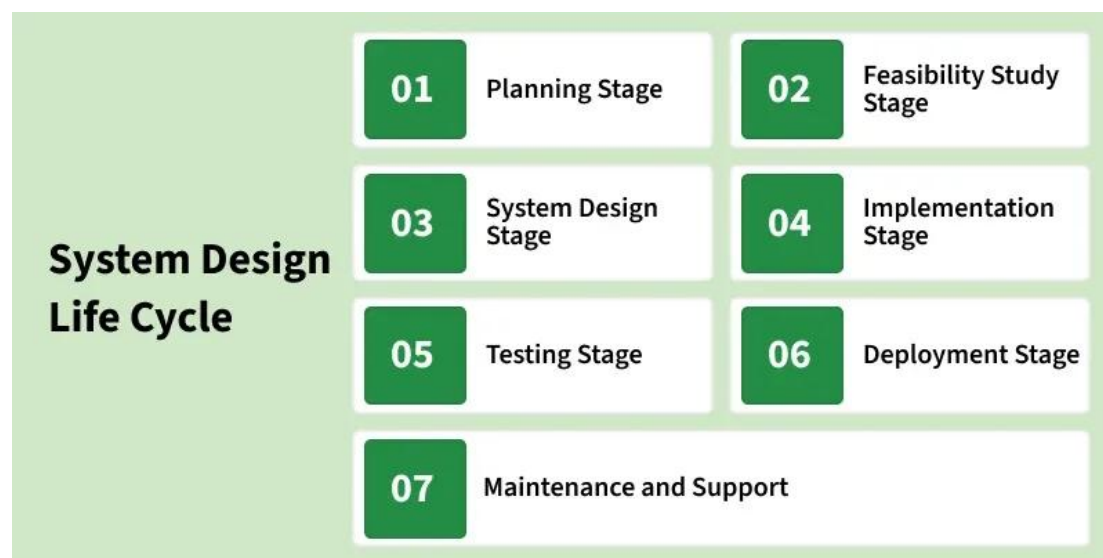
- [How to Prepare for Low-Level Design Interviews?](#)
- [Top Low-Level Design\(LLD\) Interview Questions 2024](#)
- [Best Books for Learning Low-Level Design\(LLD\) \[2024\]](#)

4. System Design Life Cycle (SDLC)

The System Design Life Cycle (SDLC) is a structured process used to plan, design, develop, test, deploy, and maintain software systems. It helps teams build reliable, scalable, and high-quality systems by following well-defined phases, ensuring that business requirements are correctly translated into technical solutions.

- Provides a clear roadmap from idea to deployment.
- Improves project planning, tracking, and control.
- Ensures better quality through systematic testing.
- Reduces development risks and cost overruns.
- Enhances collaboration between stakeholders and developers.
- Makes systems easier to maintain and scale over time.

Below is the objective and example of each stage of System Design Life Cycle:



[Stage 1]: Planning

- This is where the project starts! You decide what you want to achieve (goals), how much you can spend (budget), and who will work on it (team).

Example: A company plans to build a customer management system and outlines its key features and the team involved.

[Stage 2]: Feasibility Study

- Before starting, check if the idea is practical. Will it work technically? Is it affordable?

***Example:** The company checks if building the system is worth the cost and effort and whether it can deliver the expected benefits.*

[Stage 3]: System Design

- This stage is all about planning how the system will work behind the scenes and how it will look to users.
- Imagine designing a house, before construction begins, architects create blueprints that show where every room, pipe, and wire will go. Similarly, in system design, you create a detailed plan that developers will follow to build the system.

[Stage 4]: Implementation

- Transform the design into an operational system. Developers write the code to create the system based on the design.

***Example:** The CRM system's features, like customer profiles and dashboards, are coded*

[Stage 5]: Testing

- Verify that the system meets the specified requirements and makes sure everything works as planned.
- The CRM system undergoes various testing procedures, such as unit testing, integration testing, and user acceptance testing, to ensure its functionality, performance, and security.

[Stage 6]: Deployment

- All of the hard work concludes at the deployment phase, when the system is made accessible for real usage. After all the planning, designing, and building is finished, it's similar like launching a new store.
- This stage involves transferring the system from a testing or development environment to a production environment where actual users can access it.

[Stage 7]: Maintenance and Support

- Ensure the ongoing functionality and address any issues that arise.
- Regular updates, bug fixes, and user support for the CRM system to adapt to changing business requirements and address any emerging issues.

Differences between the System Development Life Cycle and the System Design Life Cycle

Below are the differences between System Development Life Cycle and System Design Life Cycle:

Aspect	System Development Life Cycle	System Design Life Cycle
Definition	A comprehensive framework covering the entire system development process.	A subset of the SDLC that specifically deals with designing the system.
Scope	Encompasses the entire life cycle of a system, from initiation to retirement.	Focuses primarily on the design aspects of the system.
Phases	Typically includes Planning, Analysis, Design, Implementation, Testing, Deployment, and Maintenance.	Usually includes Feasibility Study, System Analysis, System Design, Implementation, Testing, Deployment, and Maintenance.

Focus	Broad focus on the overall development process, addressing planning, implementation, testing, and maintenance.	Specific focuses on the design phase, detailing how the system will be built and operate.
Purpose	Guides the development team through the entire process, from concept to post-deployment support.	Provides a blueprint for constructing the system based on specified design requirements.

Challenges in System Design Life Cycle

Below are the challenges in System Design Life Cycle:

- Sometime, the initial requirements for a system might be unclear or ambiguous, leading to difficulties in designing the system accurately.
- Requirements may change during the design process, posing a challenge to maintain consistency and ensuring that the system still meets the user's needs.
- Rapid advancements in technology can make it challenging to choose the most suitable and up-to-date technologies for system design.
- Ensuring seamless integration of various system components can be complex, especially when dealing with different technologies and platforms.
- Designing a system within budgetary constraints can be challenging, as incorporating certain features or technologies might be cost-productive.

Models Used for System Design Life Cycle

Below are the models used for System Design Life Cycle:

- **Waterfall Model:** A linear and sequential model where each phase must be completed before moving on to the next. It's a straightforward approach but can be inflexible in the face of changing requirements.
- **Iterative Model:** Involves repeating cycles, with each iteration refining and improving the system based on feedback. It's adaptable to changing requirements.

- **Prototyping Model**: Involves building a prototype (a preliminary version) of the system to gather feedback refine the design before building the final product.
- **Spiral Model**: Incorporates elements of both iterative and prototyping models. It involves cycles of planning, designing, constructing, and evaluating.
- **Agile Model**: Emphasizes flexibility and collaboration, with frequent iterations and continuous feedback. It's well suited for projects where requirements may evolve.

Iterative nature of SDLC

The iterative nature of the System Design Life Cycle (SDLC) means the process is not strictly linear; instead, it involves revisiting and refining stages based on feedback and evolving requirements. This ensures the system remains adaptable and aligned with its objectives.

Key Aspects of Iteration in SDLC:

1. **Refining Requirements**: Stakeholders may identify additional needs or changes after deployment or testing. These insights prompt revisiting earlier phases, such as planning or design, to incorporate updates.
2. **Prototyping and Testing**: Prototypes or early iterations of the system are developed and evaluated. Better performance and user satisfaction are ensured by improving the design and implementation based on feedback from these trials.
3. **Adapting to Changes**: The system design is reviewed and modified to be relevant and efficient as user expectations, technological advancements, or corporate objectives change.

The system may gradually evolve to satisfy present and future expectations due to this iterative process, which also ensures ongoing improvement and reduces risks. It's similar to building a bridge and then modifying it over time to handle growing traffic or changing conditions.

Best Practices in System Design Life Cycle

Below are the best practices in System Design Life Cycle:

- Invest time in thoroughly understanding and documenting requirements to provide a solid foundation for the design process.

- Maintain open communication with stakeholders to ensure their needs are understood and to address any changes promptly.
- Design systems in a modular fashion, allowing for easier maintenance, updates, and scalability.
- Identify potential risks early in the design process and develop strategies to mitigate or manage them effectively.

Use Cases of System Design Life Cycle

The SDLC is used in a wide variety of projects, from developing small applications to building large enterprise systems. Some common use cases for the SDLC includes:

- Developing new software applications.
- Enhancing existing software applications.
- Integrating different systems together.
- Replacing legacy systems.
- Developing custom solutions for specific business needs.

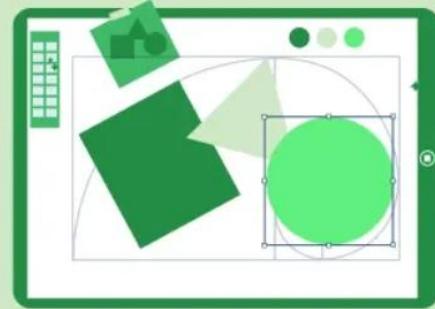
By following the best practices and using the appropriate SDLC model, organizations can increase their chances of successfully completing their system development projects.

5 Design Patterns Tutorial

Design patterns are reusable solutions to common software design problems that help developers build cleaner and more maintainable systems. They provide templates for solving issues in object creation, structure, and behaviour. They improve code flexibility and scalability.

What is Design Pattern?

- Reusable solutions to common software design problems
- Improve code structure, flexibility, and maintainability
- Provide proven templates for object creation, interaction, & architecture



1 / 7

Types of Design Pattern



**Creational Design
Patterns**



**Structural
Design Patterns**



**Behavioral Design
Patterns**

Creational Design Pattern

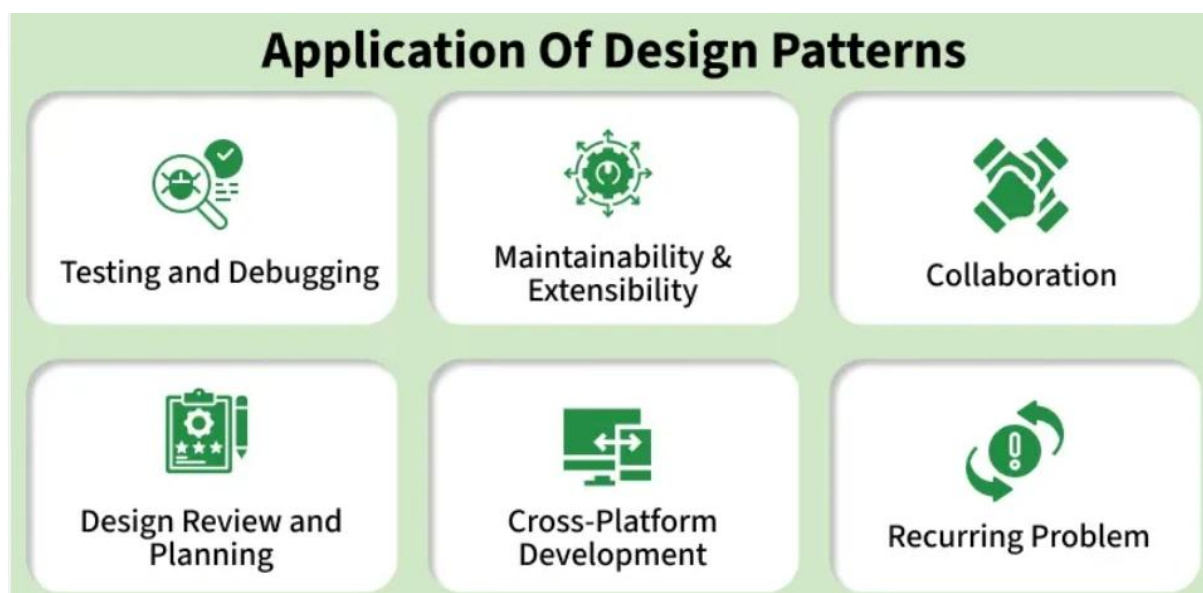
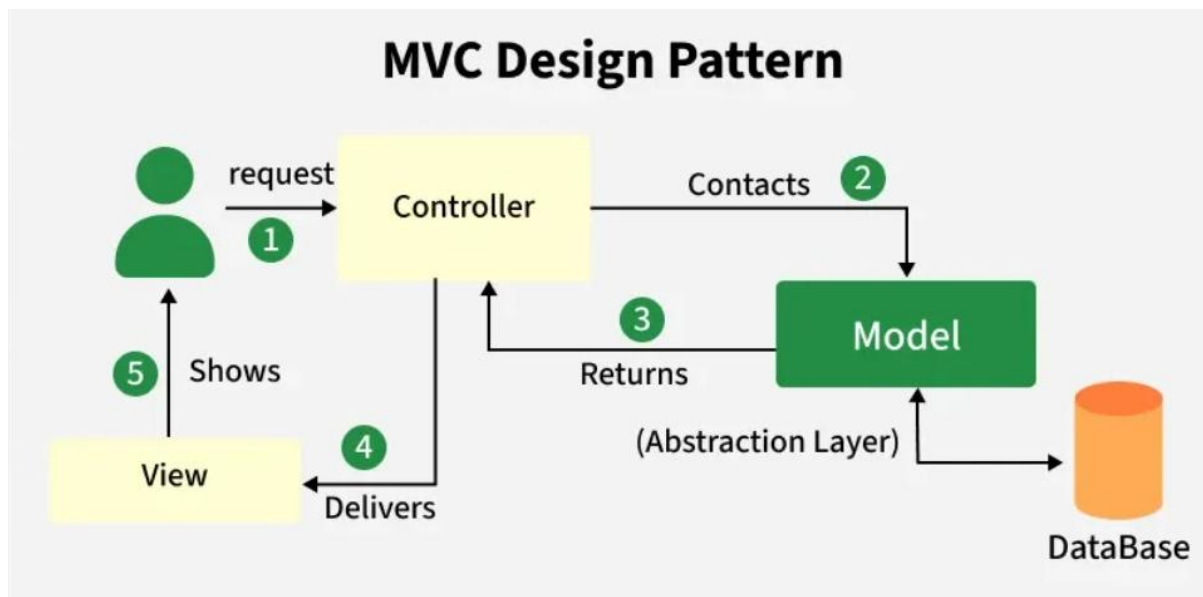
- 1 Singleton Pattern
- 2 Abstract Pattern
- 3 Factory Pattern
- 4 Prototype Pattern
- 5 Builder Pattern

Structural Design Patterns

- 01 Adapter Pattern
- 02 Bridge Pattern
- 03 Composite Pattern
- 04 decorator Pattern
- 05 Facade Pattern
- 06 Flyweight Pattern
- 07 Proxy Pattern

Behavioral Design Patterns

- | | |
|----------------------------|---------------------|
| 01 Observer Pattern | 02 Strategy Pattern |
| 03 State Pattern | 04 Command Pattern |
| 05 Chain of Responsibility | 06 Template Pattern |
| 07 Interpreter Pattern | 08 Visitor Pattern |
| 09 Mediator Pattern | 10 Memento Pattern |



- Provide standard, proven solutions to recurring design problems.
- Improve code readability, reusability, and maintainability.
- Help developers follow best practices in software architecture.
- Reduce development time by avoiding reinventing solutions.

Design Pattern Basics

Design patterns provide reusable solutions to common software design problems. They help developers write cleaner, maintainable, and scalable applications.

- [Introduction](#)
- [Types of Design Patterns](#)

- [Gang of Four \(GoF\) Overview](#)

Creational Design Patterns

These [Creational Design Patterns](#) deal with object creation in a flexible and efficient manner. They help you control how and when objects are instantiated.

- [Singleton Pattern](#)
- [Factory Method Pattern](#)
- [Abstract Factory Pattern](#)
- [Builder Pattern](#)
- [Prototype Pattern](#)
- [Object Pool Pattern](#)
- [Lazy Initialization](#)

Structural Design Patterns

[Structural patterns](#) explain how classes and objects are combined to form larger structures. They improve code flexibility by simplifying relationships between components.

- [Adapter Pattern](#)
- [Decorator Pattern](#)
- [Facade Pattern](#)
- [Composite Pattern](#)
- [Proxy Pattern](#)
- [Bridge Pattern](#)
- [Flyweight Pattern](#)

Behavioral Design Patterns

[Behavioral patterns](#) define how objects communicate and distribute responsibilities. They help manage workflows, interactions, and decision-making within a system

- [Observer Pattern](#)
- [Strategy Pattern](#)
- [Command Pattern](#)

- [Chain of Responsibility Pattern](#)
- [Template Method Pattern](#)
- [Iterator Pattern](#)
- [State Pattern](#)
- [Mediator Pattern](#)
- [Memento Pattern](#)
- [Visitor Pattern](#)

Advanced Design Pattern

Advanced topics cover architectural principles and deeper system-design concepts. They help you build enterprise-level, scalable, and robust software systems

- [SOLID Principles](#)
- [DRY Principle](#)
- [KISS Principle](#)
- [YAGNI Principle](#)
- [Dependency Injection Pattern](#)
- [Composition Vs Inheritance](#)
- [Event-Driven Architecture](#)
- [CQRS Design Pattern](#)
- [Event Sourcing Patterns](#)
- [CQRS Vs Event Sourcing Patterns](#)
- [Repository Pattern](#)
- [MVC Design Pattern](#)

Design Pattern Interview Questions

This section prepares you for common interview questions on design patterns. It strengthens conceptual clarity and helps you explain patterns confidently.

- [Interview Questions on Software Design Patterns](#)

Approach to Learn Design Patterns

- **Week 1:** You'll learn the foundations needed for design patterns, including classes, objects, abstraction, inheritance, polymorphism, and encapsulation. You will also study SOLID principles, which help you write clean, maintainable, and loosely coupled code - essential before learning design patterns.
- **Week 2:** Focuses on Creational Patterns such as Singleton, Factory Method, Abstract Factory, Builder, and Prototype. You'll understand how these patterns manage object creation, reduce code complexity, and improve flexibility.
- **Week 3:** You'll explore Adapter, Decorator, Facade, Composite, Proxy, Bridge, and Flyweight patterns. These patterns teach you how to structure classes and objects efficiently, simplifying complex systems.
- **Week 4:** Dives into Strategy, Observer, Command, Chain of Responsibility, Iterator, Mediator, State, Template Method, and more. You'll learn how objects communicate, coordinate actions, and share responsibilities in real systems.
- **Week 5:** You'll study system-level patterns like MVC, MVVM, Dependency Injection, Repository Pattern, and Service Layer Pattern. These patterns are widely used in backend systems, frameworks, and enterprise applications.
- **Week 6:** Helps you apply patterns to real situations: Logger (Singleton), Notification System (Observer), Payment Gateway (Strategy), Undo/Redo (Command), and API Facade (Facade Pattern). This week focuses fully on hands-on implementation.
- **Week 7:** Covers low-level design problems where design patterns are essential. You'll practice Parking Lot System, In-Memory File System, BookMyShow LLD, and securing Singleton from Reflection, Cloning, and Serialization.
- **Week 8:** You'll build real-world projects using multiple design patterns combined. You'll also revise interview questions, pattern comparisons, and architecture-based solutions to strengthen your problem-solving skills.

Important Links

- [Design a Parking lot using Object Oriented Principles](#)
- [Design data structures and algorithms for in-memory file system](#)

6. Unified Modeling Language (UML) Diagrams

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. UML is not a programming language, it is rather a visual language.

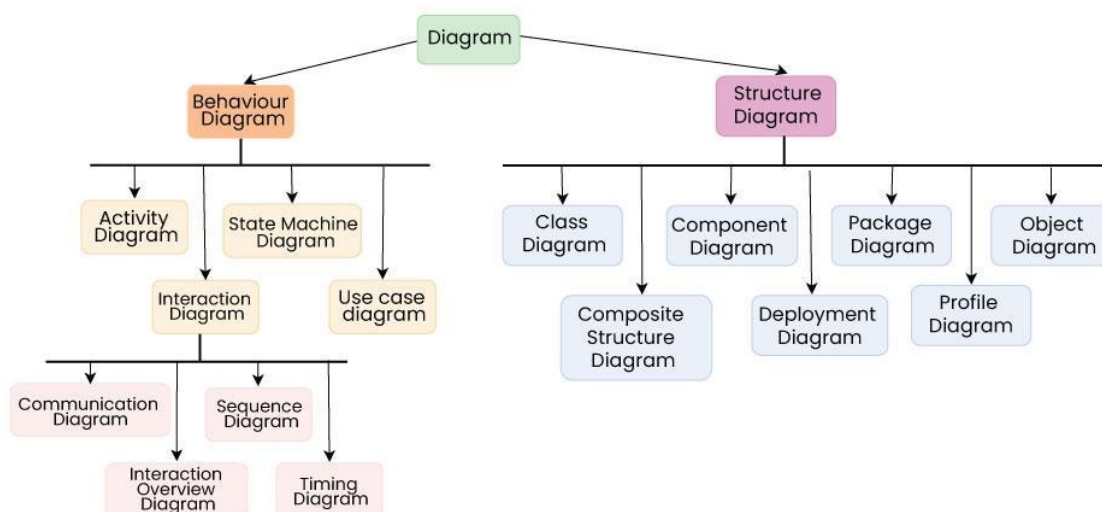
- We use UML diagrams to show the behavior and structure of a system.
- UML helps software engineers, businessmen, and system architects with modeling, design, and analysis.

Need of UML

- Complex applications need collaboration and planning from multiple teams and hence require a clear and concise way to communicate amongst them.
- Businessmen do not understand code. So UML becomes essential to communicate with non-programmers about essential requirements, functionalities, and processes of the system.
- A lot of time is saved down the line when teams can visualize processes, user interactions, and the static structure of the system.

Types of UML Diagrams

UML is linked with object-oriented design and analysis. UML makes use of elements and forms associations between them to form diagrams. Diagrams in UML can be broadly classified as:

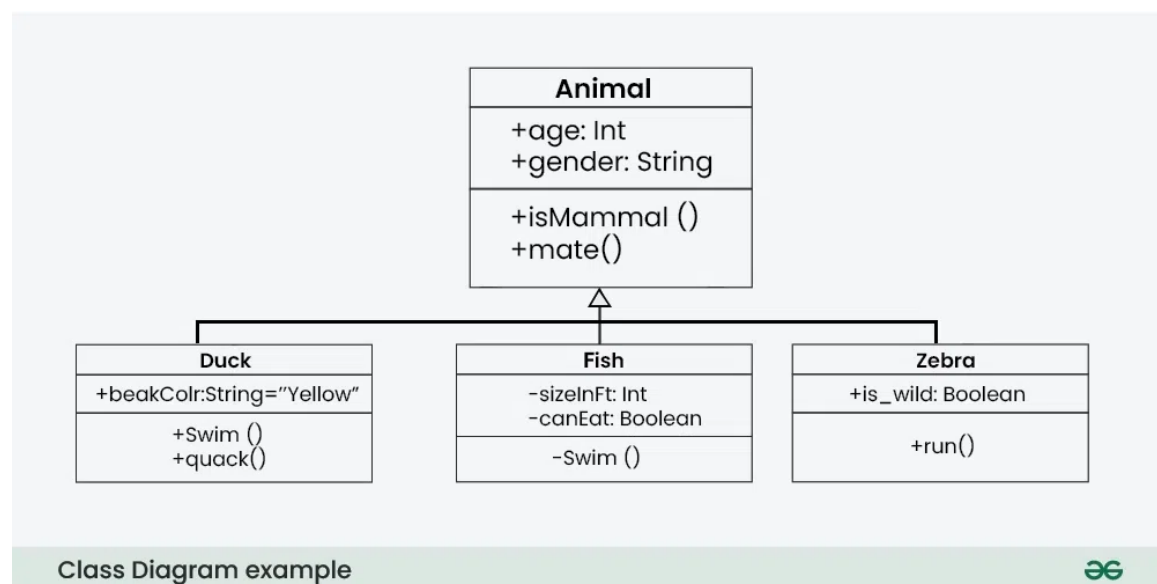


Structural UML Diagrams

[Structural UML diagrams](#) are visual representations that depict the static aspects of a system, including its classes, objects, components, and their relationships, providing a clear view of the system's architecture. Structural UML diagrams include the following types:

Class Diagram

The most widely use UML diagram is the class diagram. It is the building block of all object oriented software systems. We use class diagrams to depict the static structure of a system by showing system's classes, their methods and attributes. Class diagrams also help us identify relationship between different classes or objects.



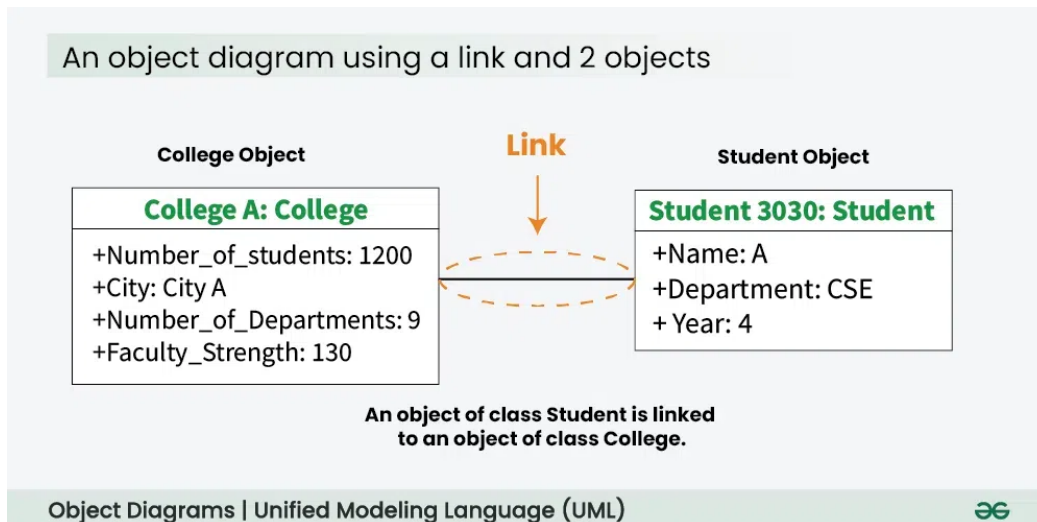
Class Diagram

Composite Structure Diagram

We use composite structure diagrams to represent the internal structure of a class and its interaction points with other parts of the system.

- A composite structure diagram represents relationship between parts and their configuration which determine how the classifier (class, a component, or a deployment node) behaves.
- They represent internal structure of a structured classifier making the use of parts, ports, and connectors.
- They are similar to class diagrams except they represent individual parts in detail as compared to the entire class.

Object Diagram



An [Object Diagram](#) can be referred to as a screenshot of the instances in a system and the relationship that exists between them. Since object diagrams depict behaviour when objects have been instantiated, we are able to study the behavior of the system at a particular instant.

- An object diagram is similar to a class diagram except it shows the instances of classes in the system.
- We depict actual classifiers and their relationships making the use of class diagrams.
- On the other hand, an Object Diagram represents specific instances of classes and relationships between them at a point of time.

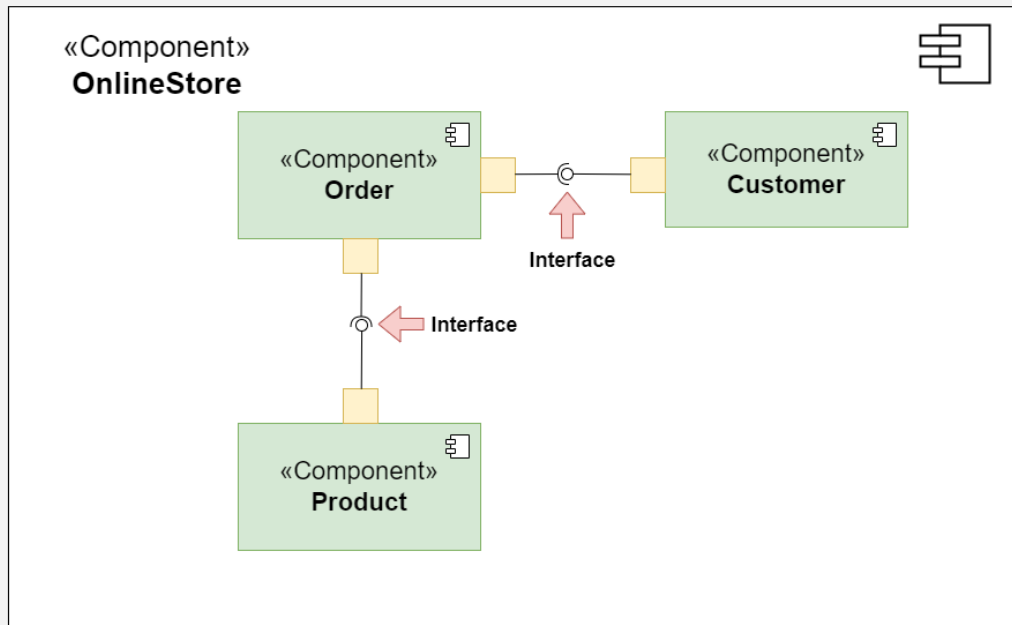
Object Diagram

Component Diagram

[Component diagrams](#) are used to represent how the physical components in a system have been organized. We use them for modelling implementation details.

- Component Diagrams depict the structural relationship between software system elements and help us in understanding if functional requirements have been covered by planned development.
- Component Diagrams become essential to use when we design and build complex systems.
- Interfaces are used by components of the system to communicate with each other.

Interfaces in Component-Based Diagram



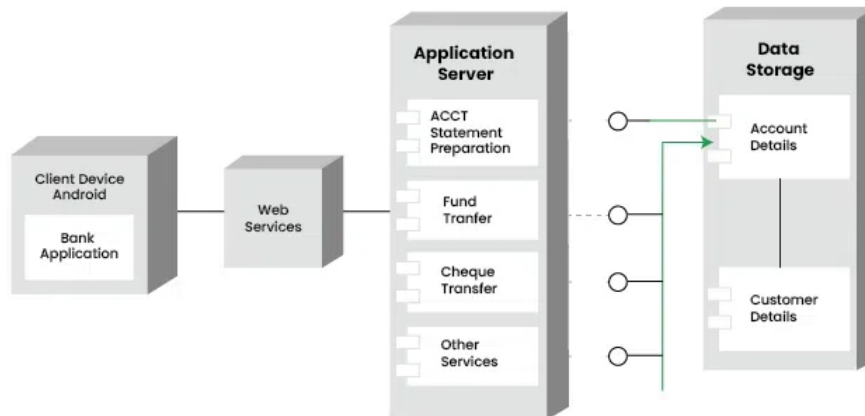
Component Diagram

Deployment Diagram

[Deployment Diagrams](#) are used to represent system hardware and its software. It tells us what hardware components exist and what software components run on them.

- We illustrate system architecture as distribution of software artifacts over distributed targets.
- An artifact is the information that is generated by system software.
- They are primarily used when a software is being used, distributed or deployed over multiple machines with different configurations.

Deployment Diagram for Mobile Banking Android Services

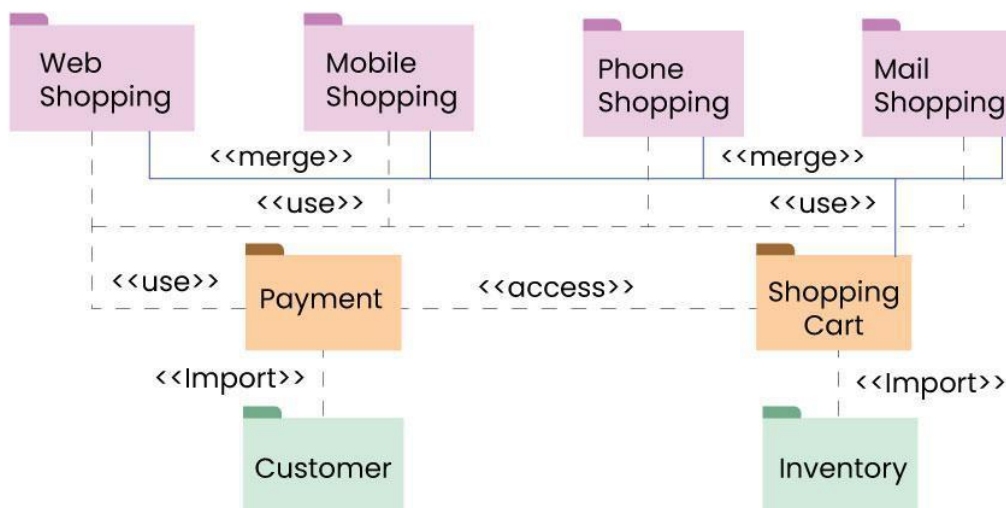


Deployment Diagram

Package Diagram

We use [Package Diagrams](#) to depict how packages and their elements have been organized. A package diagram simply shows us the dependencies between different packages and internal composition of packages.

- Packages help us to organize UML diagrams into meaningful groups and make the diagram easy to understand.
- They are primarily used to organize class and use case diagrams.



Package Diagram

Package Diagram

Behavioral UML Diagrams

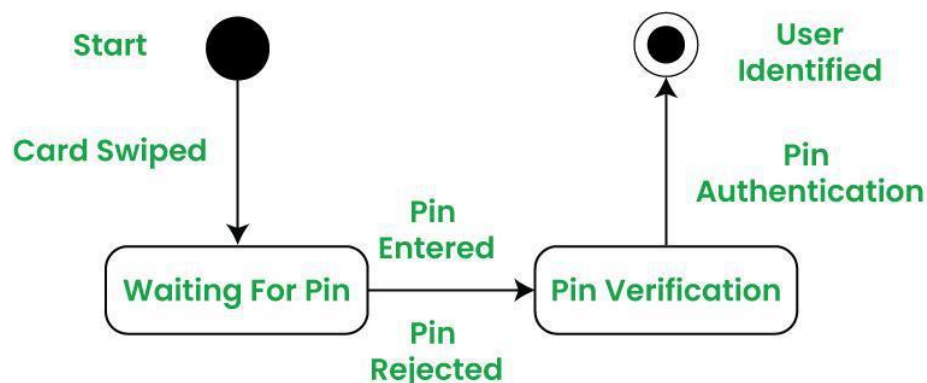
[Behavioral UML diagrams](#) are visual representations that depict the dynamic aspects of a system, illustrating how objects interact and behave over time in response to events.

State Machine Diagrams

A [state diagram](#) is used to represent the condition of the system or part of the system at finite instances of time. It's a behavioral diagram and it represents the behavior using finite state transitions.

- State diagrams are also referred to as **State machines** and **State-chart Diagrams**
- These terms are often used interchangeably. So simply, a state diagram is used to model the dynamic behavior of a class in response to time and changing external stimuli.

A State Machine Diagram for user verification



State Machine Diagrams | Unified Modeling Language (UML)



State Machine Diagram

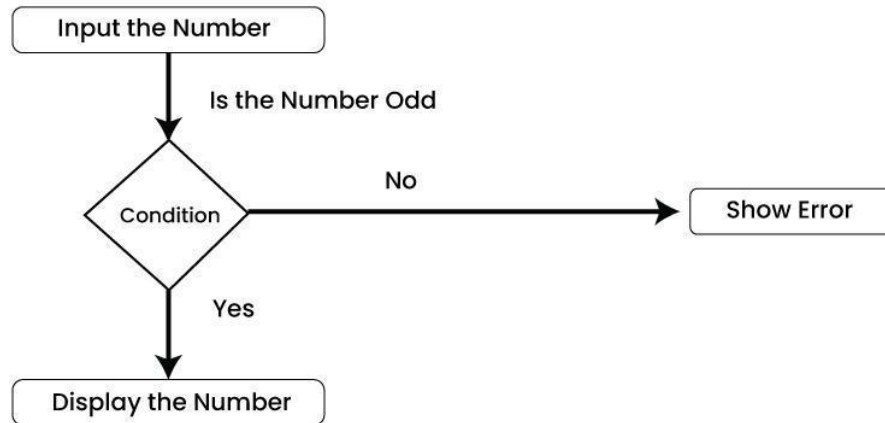
Activity Diagrams

We use [Activity Diagrams](#) to illustrate the flow of control in a system. We can also use an activity diagram to refer to the steps involved in the execution of a use case.

- We model sequential and concurrent activities using activity diagrams. So, we basically depict workflows visually using an activity diagram.
- An activity diagram focuses on condition of flow and the sequence in which it happens.

- We describe or depict what causes a particular event using an activity diagram.

An Activity Diagram using Decision Node



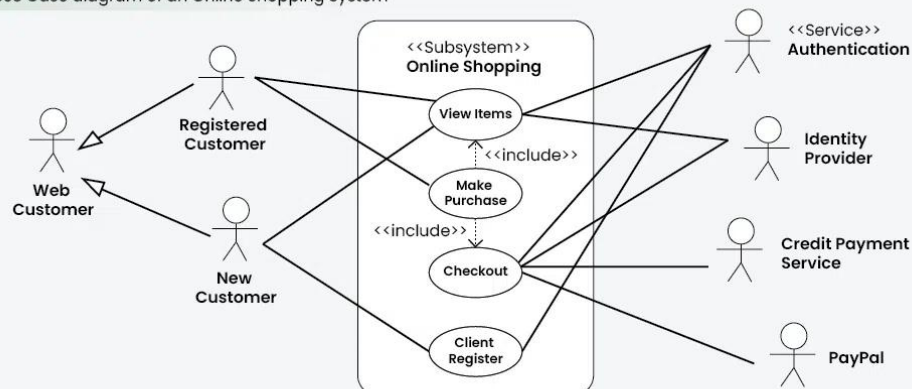
Activity Diagram

Use Case Diagrams

Use [Case Diagrams](#) are used to depict the functionality of a system or a part of a system. They are widely used to illustrate the functional requirements of the system and its interaction with external agents(actors).

- A use case is basically a diagram representing different scenarios where the system can be used.
- A use case diagram gives us a high level view of what the system or a part of the system does without going into implementation details. '

Use Case diagram of an Online Shopping System

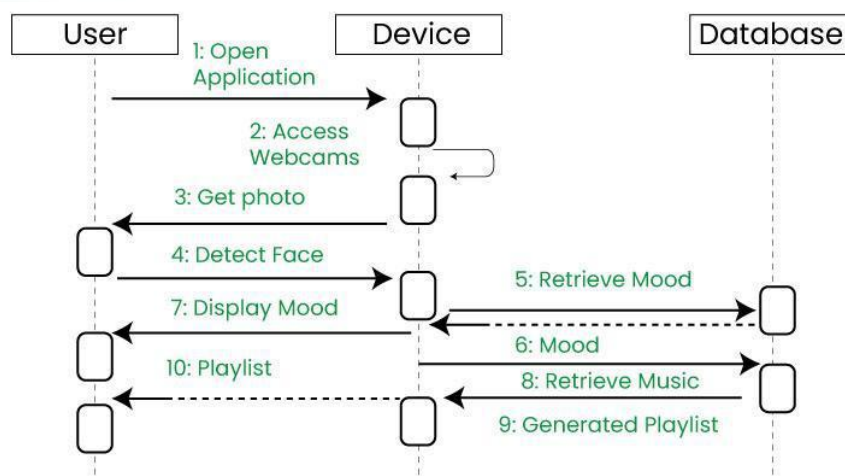


Sequence Diagram

A [sequence diagram](#) simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place.

- We can also use the terms event diagrams or event scenarios to refer to a sequence diagram.
- Sequence diagrams describe how and in what order the objects in a system function.
- These diagrams are widely used by businessmen and software developers to document and understand requirements for new and existing systems.

Example sequence diagram



Sequence Diagrams

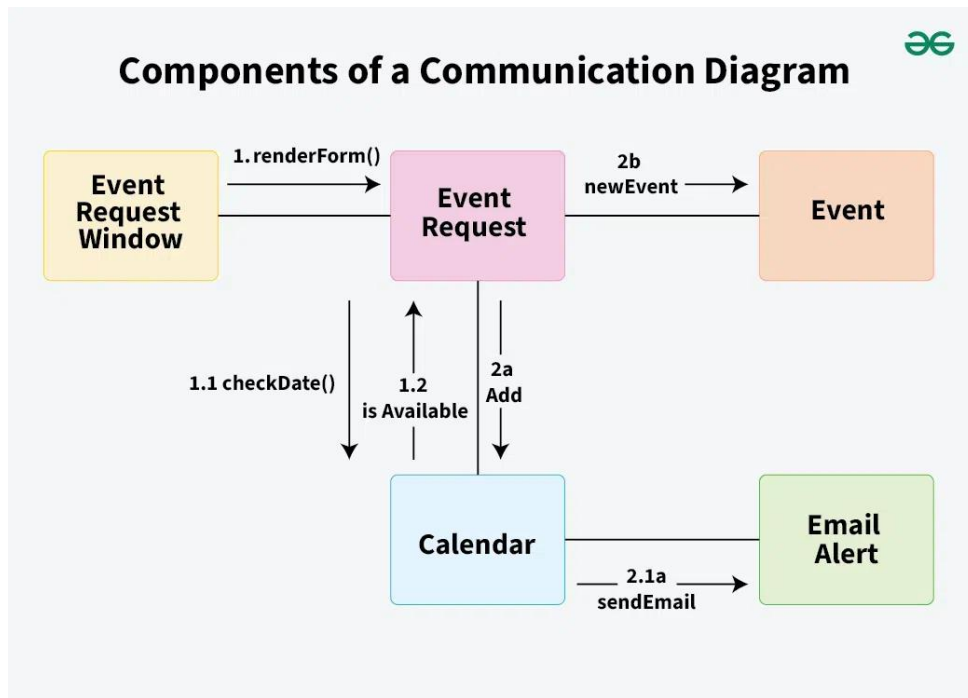


Sequence Diagram

Communication Diagram

A [Communication Diagram](#) (known as Collaboration Diagram in UML 1.x) is used to show sequenced messages exchanged between objects.

- A communication diagram focuses primarily on objects and their relationships.
- We can represent similar information using Sequence diagrams, however communication diagrams represent objects and links in a free form.



Communication Diagram

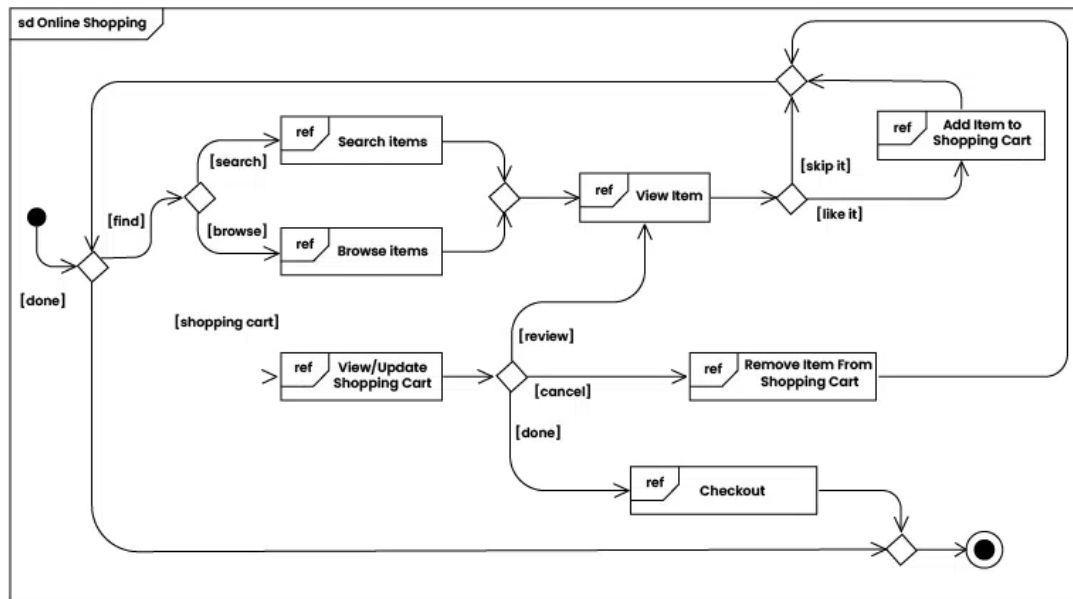
Timing Diagram

[Timing Diagram](#) are a special form of Sequence diagrams which are used to depict the behavior of objects over a time frame. We use them to show time and duration constraints which govern changes in states and behavior of objects.

Interaction Overview Diagram

An [Interaction Overview Diagram \(IOD\)](#) is a type of UML (Unified Modeling Language) diagram that illustrates the flow of interactions between various elements in a system or process. It provides a high-level overview of how interactions occur, including the sequence of actions, decisions, and interactions between different components or objects.

Example of Interaction overview Diagram



Interaction Overview Diagram

Additions in UML 2.0

- Software development methodologies like agile have been incorporated and scope of original UML specification has been broadened.
- Originally UML specified 9 diagrams. UML 2.x has increased the number of diagrams from 9 to 13. The four diagrams that were added are : timing diagram, communication diagram, interaction overview diagram and composite structure diagram. UML 2.x renamed statechart diagrams to state machine diagrams.
- UML 2.x added the ability to decompose software system into components and sub-components.

Tools for creating UML Diagrams

There are several tools available for creating Unified Modeling Language (UML) diagrams, which are commonly used in software development to visually represent system architecture, design, and implementation. Here are some popular UML diagram creating tools:

- **Lucidchart:** Lucidchart is a web-based diagramming tool that supports UML diagrams. It's user-friendly and collaborative, allowing multiple users to work on diagrams in real-time.

- **Draw.io:** Draw.io is a free, web-based diagramming tool that supports various diagram types, including UML. It integrates with various cloud storage services and can be used offline.
- **Visual Paradigm:** Visual Paradigm provides a comprehensive suite of tools for software development, including UML diagramming. It offers both online and desktop versions and supports a wide range of UML diagrams.
- **StarUML:** StarUML is an open-source UML modeling tool with a user-friendly interface. It supports the standard UML 2.x diagrams and allows users to customize and extend its functionality through plugins.

You can also refer to the link - [Steps to create UML Diagram](#)
For steps to create uml diagram

7. Scalability in System Design

Scalability refers to a system's ability to grow smoothly and handle increased demand while maintaining performance, reliability, and efficiency.

- Handles rising user traffic and workload effectively.
- Supports growth in data and computing needs.
- Maintains performance under increased load.
- Avoids major redesign during expansion.

Importance of Scalability in System Design

Scalability is an important factor in system design, which offers many benefits such as:

- **Managing Growth:** Handles more users, data, and traffic without losing speed or reliability.
- **Improving Performance:** Distributes load across resources for faster processing and responses.
- **Ensuring Availability:** Keeps systems running during traffic spikes or component failures.
- **Cost-effectiveness:** Scales resources up or down as needed, reducing unnecessary costs.

- **Encouraging Innovation:** Makes it easier to add new features and adapt to market changes.

How to achieve Scalability?

Make It Bigger ([Vertical Scaling](#))

- Like upgrading a car with a bigger engine for more power.
- In tech, it means adding CPU, memory, or storage to the same server.
- Suitable for small applications and quick scaling needs.
- Limited by hardware, as upgrades can't continue indefinitely.

Get More Cars ([Horizontal Scaling](#))

- Like using multiple cars to share the workload.
- Adds more servers or instances instead of upgrading one.
- Distributes traffic evenly across resources.
- Ideal for large applications with many users.

Divide and Conquer ([Microservices](#))

- Treats the app as small, independent services.
- Scales only the required parts instead of the whole system.
- Improves flexibility and efficient resource usage.

No Servers, No Problems ([Serverless](#))

- Removes the need to manage servers.
- Automatically scales based on demand.
- Cost-efficient for variable and unpredictable workloads.
- Example : AWS Lambda

Factors Affecting Scalability

Below are the factors that affects the scalability with their explanation:

Performance Bottlenecks:

- Performance bottlenecks are parts of a system that slow down overall performance.

- They are often caused by slow databases, inefficient code, or limited resources.

Resource Utilization:

- Efficiently using resources such as CPU, memory, and disk space is essential for scalability.
- Inefficient resource utilization can lead to bottlenecks and limit the system's ability to scale.

Network Latency:

- Network latency is the delay in data transmission.
- High latency slows node communication and affects scalability.

Data Storage and Access:

- Data storage and access patterns affect scalability.
- Distributed databases and caching help systems scale better.

Concurrency and Parallelism:

- Concurrency and parallelism improve scalability by handling multiple tasks at the same time.
- They increase throughput by processing more requests simultaneously.
- They reduce response time, making systems faster and more efficient.

System Architecture:

- System architecture affects how easily a system can scale.
- Modular and loosely coupled components improve scalability.
- Such architectures support horizontal (adding instances) and vertical (upgrading resources) scaling.

Components that help to increase Scalability

Below are some of the main components that help to increase the scalability:

- **Load Balancer:** A [load balancer](#) distributes incoming traffic across multiple servers to avoid overload and improve performance and availability.
- **Caching:** [Caching](#) stores frequently accessed data temporarily to reduce latency and backend load.

- **Database Replication:** [Database replication](#) creates multiple real-time copies of data to enhance availability and read performance.
- **Database Sharding:** [Database sharding](#) splits data into smaller shards to scale databases across multiple instances.
- **Microservices Architecture:** [Microservices architecture](#) divides applications into independent services that can scale separately.
- **Data Partitioning:** [Data partitioning](#) divides data based on criteria like user or region to improve scalability.
- **Content Delivery Networks (CDNs):** [CDNs](#) deliver cached content from locations closer to users, reducing latency.
- **Queueing Systems:** [Queueing systems](#) handle requests asynchronously to manage traffic spikes and prevent overload.

Real-World Examples of Scalable Systems

There are many real-world examples of scalable systems that demonstrate the importance and impact of scalability in modern technology.

- **Google:** Uses a highly scalable distributed system (Bigtable, MapReduce, Spanner) to handle billions of searches globally.
- **AWS:** Offers scalable cloud services that let businesses easily scale compute, storage, and databases on demand.
- **Netflix:** Relies on cloud infrastructure, microservices, and caching to stream content to millions of users at once.

These examples highlight how scalability helps companies deliver reliable performance, handle traffic spikes, and grow rapidly without infrastructure limitations.

Challenges and Trade-offs in Scalability

Challenges and trade-offs include:

- **Cost vs. Scalability:** Scaling improves performance and availability but often increases infrastructure and operational costs.
- **Complexity:** As systems scale, they become harder to manage, maintain, and debug, raising operational overhead.

- **Latency vs. Throughput:** There is often a trade-off between [latency and throughput](#). Optimizing for low latency may reduce throughput, and vice versa.
- **Data Partitioning Trade-offs:** [Partitioning](#) boosts scalability but requires careful balance of partition size, data movement, and data locality.

8. Software Engineering Tutorial

Software Engineering is the process of designing, developing, testing, and maintaining software.

- It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.
- It includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.



Basics

Learn what software is and why Software Engineering is needed.

1. [Introduction](#)
2. [Classification of Software](#)
3. [Software Evolution](#)
4. [Need of Software Engineering](#)

Software Development Models

Need to manage complexity, reduce risks, and ensure software is built systematically, on time, and within budget.

1. [Classical Waterfall Model](#)
2. [Iterative Waterfall Model](#)
3. [Spiral Model](#)
4. [Incremental process model](#)
5. [Rapid application development model\(RAD\)](#)
6. [RAD Model vs Traditional SDLC](#)
7. [Agile Development Models](#)
8. [Agile Software Development](#)
9. [Extreme Programming \(XP\)](#)
10. [SDLC V-Model](#)
11. [Comparison of different life cycle models](#)

Software Architecture

Defines the high-level structure of a system, showing how components are organized and interact with each other.

1. [User Interface Design](#)
2. [Coupling and Cohesion](#)
3. [Information System Life Cycle](#)
4. [Database application system life cycle](#)
5. [Pham-Nordmann-Zhang Model \(PNZ model\)](#)
6. [Schick-Wolverton software reliability model](#)

Software Project Management(SPM)

Focuses on planning, organizing, and controlling software projects to deliver quality software on time and within budget.

1. [Project Management Process](#)
2. [Project size estimation techniques](#)

3. [System configuration management](#)
4. [COCOMO Model](#)
5. [Capability maturity model \(CMM\)](#)
6. [Integrating Risk Management in SDLC | Set 1](#)
7. [Integrating Risk Management in SDLC | Set 2](#)
8. [Integrating Risk Management in SDLC | Set 3](#)
9. [Role and Responsibilities of a software Project Manager](#)
10. [Software Project Management Complexities](#)
11. [Quasi renewal processes](#)
12. [Reliability Growth Models](#)
13. [Jelinski Moranda software reliability model](#)
14. [Schick-Wolverton software reliability model](#)
15. [Goel-Okumoto Model](#)
16. [Mills' Error Seeding Model](#)
17. [Basic fault tolerant software techniques](#)
18. [Software Maintenance](#)

Software Metrics

Measurable values used to evaluate the quality, performance, and efficiency of software and its development process.

1. [Software Measurement and Metrics](#)
2. [People Metrics and Process Metrics in Software Engineering](#)
3. [Halstead's Software Metrics](#)
4. [Cyclomatic Complexity](#)
5. [Functional Point \(FP\) Analysis – Software Engineering](#)
6. [Lines of Code \(LOC\) in Software Engineering](#)

Software Requirements

Define what a system must do and serve as the foundation for its design, development, and testing.

1. [Requirements Engineering Process](#)
2. [Classification of Software Requirements](#)
3. [How to write a good SRS for your Project](#)
4. [Quality Characteristics of a good SRS](#)
5. [Requirements Elicitation](#)
6. [Challenges in eliciting requirements](#)

Software Configuration

Focuses on managing and controlling changes to software components and artifacts throughout the development lifecycle.

1. [Software Configuration Management](#)
2. [Objectives of Software Configuration Management](#)
3. [Software Quality Assurance](#)
4. [Project Monitoring & Control](#)

Software Quality

Measures how well a software product meets requirements and delivers reliable, efficient, and user-friendly performance.

1. [Software Quality](#)
2. [ISO 9000 Certification](#)
3. [SEICMM](#)
4. [Six Sigma](#)

Software Design

Creates a structured blueprint that defines how a software system will meet its requirements efficiently.

1. [Software Design Process](#)
2. [Software Design process – Set 2](#)
3. [Software Design Principles](#)
4. [Coupling and Cohesion](#)
5. [Function Oriented Design](#)

6. [Object Oriented Design](#)
7. [User Interface Design](#)

Software Reliability

Measures how consistently a system performs its intended functions without failure over time.

1. [Software Reliability](#)
2. [Software Fault Tolerance](#)

Software Testing and Debugging

It ensure that software works correctly by identifying and fixing defects throughout development.

1. [Software Testing Tutorial](#)
2. [Seven Principles of software testing](#)
3. [Testing Guidelines](#)
4. [Black box testing](#)
5. [White box Testing](#)
6. [Debugging](#)
7. [Selenium: An Automation tool](#)
8. [Integration Testing](#)

Software Maintenance

Involves updating and improving software to keep it effective, efficient, and relevant over time.

1. [Software Maintenance](#)
2. [Cost and efforts of software maintenance](#)

Difference Between

Understanding the differences between software engineering concepts helps choose the right approach for building effective and efficient software.

1. [Waterfall model vs Incremental model](#)
2. [v-model vs waterfall model](#)
3. [Manual testing vs Automation testing](#)

4. [Sanity Testing vs Smoke Testing](#)
5. [Cohesion vs Coupling](#)
6. [Alpha Testing vs Beta Testing](#)
7. [Testing and Debugging](#)
8. [Functional vs Non-functional Testing](#)
9. [Waterfall Model vs Spiral Model](#)
10. [RAD vs Waterfall](#)
11. [Unit Testing vs System Testing](#)
12. [Load Testing vs Stress Testing](#)
13. [Frontend Testing vs Backend Testing](#)
14. [Agile Model vs V-Model](#)

Software Engineering Interview Questions

1. [Software Engineering Interview Questions](#)
2. [Software Engineering Quiz](#)
3. [SDLC MCQ Questions](#)
4. [Risk Management MCQ Questions](#)
5. [Software Development Models MCQ Questions and Answers](#)
6. [Software Quality Assurance Quiz](#)